UPTD BALAI LATIHAN KERJA (BLK) PROVINSI GORONTALO

Pelatihan Berbasis Kompetensi

JUNIOR WEB DEVELOPER

MODUL KERJA 4F

Fondasi Backend dan Berkas Data

Bagian B2: PHP, Berkas JSON, Composer (Jalur Alternatif Tanpa Basis Data)

Batch #5 · Kelas: Junior Web Developer

Instruktur: Rahmat S.A.F. Badaru

2026

Daftar Isi

- A. Identitas Modul
- B. Pemetaan Unit Kompetensi dan Kriteria Capaian
- C. Tujuan Pembelajaran
- D. Petunjuk Belajar
- Kegiatan Belajar 4F.1: Bersih-Bersih Proyek dan Dasar PHP (Hari 12)
- Kegiatan Belajar 4F.2: Baca-Tulis JSON dan Proteksi Folder Data (Hari 13)
- Kegiatan Belajar 4F.3: Migrasi Frontend ke API (Hari 14)
- Kegiatan Belajar 4F.4: Penguncian Berkas dan Persiapan Transaksi (Hari 15)
- E. Lembar Kerja Pendamping
- F. Lembar Asesmen Modul 4F
- G. Glosarium
- H. Referensi
- Lampiran A: Kode Acuan Lengkap
- Lampiran B: Rancangan Struktur Berkas Data Setelah Modul 4F

A. Identitas Modul

Komponen	**Keterangan**
Nama modul	Fondasi **Backend** dan Berkas Data
Bagian	B2: PHP, Berkas JSON, Composer — **jalur alternatif** Bagian B, berdampingan (bukan pengganti)
Durasi	32 JP (1 JP = 45 menit), 4 hari × 8 JP, Hari 12-15
Prasyarat	Bagian A selesai (Modul 1-3): proyek Kopi Nusantara Versi 3 (fetch + JSON) dan proyek tema sendiri peserta, keduanya berjalan di Laragon
Luaran	(1) Tiga temuan Bagian A diperbaiki; (2) folder `data/` terlindungi dari akses langsung lewat browser; (3) endpoint `api/produk.php` yang membaca `data/produk.json` di sisi server; (4) `app.js` Kopi Nusantara sudah mengambil data lewat API, bukan lagi langsung dari berkas; (5) rancangan `transaksi.json` dan `pengguna.json` siap dipakai Modul 5F

Komponen	Keterangan
Perangkat	Komputer/laptop, Laragon (Apache + PHP 8.5 aktif — **tidak perlu MariaDB** untuk jalur ini)
Perangkat lunak	VS Code, browser dengan DevTools

Gambaran modul. Bagian A **berhenti** di satu batas **yang disengaja**: data produk memang sudah "dinamis" secara tampilan, tapi sumbernya tetap berkas statis **(`data/produk.json`)** yang **langsung dibaca browser** lewat `fetch()`. Modul ini **memindahkan tugas** membaca (dan nantinya menulis) berkas itu ke **PHP di server** — bukan dengan berpindah ke basis data, melainkan dengan tetap memakai berkas JSON, kini dikelola dengan **lebih hati-hati: terlindungi dari akses langsung**, dan (mulai Hari ke-15) ditulis dengan mekanisme penguncian yang aman.

Hari	Fokus	Luaran
11	Bersih-bersih proyek Bagian A, dasar PHP	Tiga bug diperbaiki, PHP pertama berjalan
12	Baca-tulis JSON, proteksi folder `data/`	`api/produk.php` berjalan, `data/` tidak bisa diakses langsung
13	Migrasi `app.js` ke API	Kopi Nusantara jalan lewat PHP, bukan fetch langsung ke berkas
14	Penguncian berkas, rancangan `transaksi.json`/`pengguna.json`	Fondasi siap untuk Modul 5F

B. Pemetaan Unit Kompetensi dan Kriteria Capaian

Modul ini **melanjutkan** lima UK yang sama seperti Bagian A dan Bagian B — bukan UK baru. Karena Bagian B2 berdampingan dengan Bagian B (memakai jatah JP yang sama, bukan tambahan), pemetaan JP per UK di bawah ini **identik** dengan Modul 4 versi basis data — hanya konteksnya yang berbeda: berkas, bukan tabel.

UK	Kode Unit	Nama Unit Kompetensi	JP di Modul 4F
1.1	J.620100.005.02	Mengimplementasikan user interface	2
1.2	J.620100.010.01	Menerapkan perintah eksekusi bahasa pemrograman	8
1.3	J.620100.015.01	Menyusun fungsi, file, atau sumber daya pemrograman dalam organisasi yang rapi	8
1.4	J.620100.016.01	Menulis kode dengan prinsip sesuai *guidelines* dan *best practices*	4
1.5	J.620100.017.02	Mengimplementasikan pemrograman terstruktur	10
		Jumlah	**32**

Kode	Kriteria capaian	Konteks di Modul 4F	Dilatihkan di
1.1-K3	Mengimplementasikan rancangan UI menjadi proses sungguhan	Tampilan publik tetap identik setelah sumber data berganti	KB 4F.3

Kode	Kriteria capaian	Konteks di Modul 4F	Dilatihkan di
1.2-K1	Cara dan tools eksekusi diidentifikasi	Menjalankan PHP lewat server (beda dari JS di browser)	KB 4F.1
1.2-K2	Source code dieksekusi sesuai mekanisme tools	PHP dieksekusi Apache/Laragon sebelum sampai ke browser	KB 4F.1, 4F.2
1.2-K3	Hasil eksekusi diidentifikasi sesuai skenario	Membaca hasil lewat browser, Elements/Network DevTools, dan log galat	KB 4F.1, 4F.2, 4F.3
1.3-K1	Nama sumber daya sesuai konteks	Nama berkas JSON, folder, dan endpoint API	KB 4F.2, 4F.4
1.3-K2	Source code dilengkapi komentar	Komentar pada PHP dan `.htaccess`	KB 4F.1 - 4F.4
1.3-K3	Folder dan subfolder tersusun sesuai isinya	Folder `api/`, `api/lib/`, `data/`	KB 4F.2
1.3-K4	README dibuat sesuai struktur folder	`README.md` diperbarui mencakup backend berbasis berkas	KB 4F.4
1.4-K1	Source code sesuai guidelines dan best practices	Folder `data/` diproteksi; penguncian berkas saat menulis	KB 4F.2, 4F.4
1.4-K2	Struktur program terorganisasi	Fungsi baca/tulis JSON dipusatkan di satu berkas (`json_store.php`)	KB 4F.2, 4F.4
1.5-K1	Program baca tulis sederhana	Membaca berkas JSON lewat PHP, menulis ulang dengan aman	KB 4F.2
1.5-K2	Percabangan dan perulangan	Menelusuri array hasil `json_decode`	KB 4F.1, 4F.2
1.5-K3	Prosedur dan fungsi	Fungsi `bacaJson()`/`tulisJson()`, fungsi pembuat kode transaksi	KB 4F.1, 4F.4
1.5-K4	Array	Array asosiatif PHP sebagai bentuk data utama	KB 4F.1, 4F.2
1.5-K5	**Program mengakses data dari media penyimpanan**	**Inti modul ini**: berkas JSON yang dikelola PHP di server, bukan lagi dibaca langsung oleh browser	KB 4F.2
1.5-K6	Memperbaiki permasalahan pada program	Menangani berkas yang belum ada, JSON tidak valid, dan risiko tulis bersamaan	KB 4F.2, 4F.4

C. Tujuan Pembelajaran

Setelah menyelesaikan modul ini, Anda mampu:

1. Menjelaskan perbedaan antara "berkas JSON dibaca langsung oleh browser" (Bagian A) dan "berkas JSON dikelola oleh PHP di server" (Bagian B2).
2. Menulis sintaks dasar PHP: variabel, array asosiatif, percabangan, perulangan, dan fungsi.
3. Membaca dan menulis berkas JSON dari PHP memakai `file_get_contents()`, `file_put_contents()`, `json_decode()`, dan `json_encode()`.
4. Melindungi folder data dari akses langsung lewat browser memakai `.htaccess`.
5. Memusatkan logika baca-tulis berkas ke dalam fungsi yang dipakai berulang, menghindari kode yang bercabang-cabang.
6. Memindahkan sumber data `app.js` dari berkas statis ke API tanpa mengubah tampilan yang sudah berfungsi.

7. Menjelaskan mengapa menulis berkas butuh penguncian (*file locking*), dan risiko apa yang dicegahnya.

D. Petunjuk Belajar

- Modul ini **tidak memerlukan MariaDB maupun phpMyAdmin** — cukup Apache dan PHP aktif di Laragon.
- PHP tetap **wajib dijalankan lewat Laragon** (alamat `http://localhost/...`). Membuka berkas `.php` langsung dari folder tidak akan berfungsi.
- Setiap mengubah kode PHP atau `.htaccess`: simpan, lalu **segarkan browser**.
- Pesan galat PHP biasanya tampil langsung di halaman browser. Baca dari atas — biasanya menyebutkan nama berkas dan nomor baris.
- Kode acuan lengkap ada di **Lampiran A**. Coba dulu sendiri sebelum membandingkan.

Kegiatan Belajar 4F.1: Bersih-Bersih Proyek dan Dasar PHP

Hari 12 · 8 JP · UK 1.2, 1.3, dan 1.5

> 💡 **Mohon dibaca dengan teliti seluruh isi Modul ya Tim?!** 😇🙏

Tujuan KB 4F.1. Setelah mengikuti KB ini, Anda mampu memperbaiki galat kecil pada proyek Bagian A, menjelaskan perbedaan PHP dan JavaScript, dan menulis sintaks dasar PHP.

Kegiatan	**JP**
Peninjauan dan perbaikan tiga temuan di proyek Kopi Nusantara	2
PHP vs JavaScript: cara kerja, cara menjalankan	1
Sintaks dasar PHP: tag, variabel, tipe data, array asosiatif	2
Percabangan, perulangan, dan fungsi di PHP	2
Latihan dan cek kemampuan	1
Jumlah	**8**

Praktik 1: Membersihkan Proyek Kopi Nusantara

> ⚠ ***Abaikan jika "Proyek Bagian A" – Anda sudah aman dan tidak ada Error lagi.***

Sebelum menambah apa pun, proyek yang sudah "selesai" tetap perlu diperiksa ulang. Buka kembali proyek Kopi Nusantara Bagian A dan perbaiki tiga hal berikut.

Temuan 1 — baris tertinggal di `js/app.js`. Buka `js/app.js`, gulir ke **baris paling bawah**. Anda akan menemukan:

```
// --- INISIALISASI ---
document.getElementById("tahun").textContent = new Date().getFullYear();
```

Baris ini adalah sisa dari pola Modul 2 (sebelum proyek memakai `fetch`). Baris yang **benar** sudah ada di dalam fungsi `mulai()`, tepat setelah `inisialisasiElemen();`. Baris di paling bawah berkas ini **berjalan sebelum data dimuat**, saat `<span id="tahun">` sudah diganti pesan "Memuat data…", sehingga `getElementById("tahun")` menghasilkan `null` dan baris ini gagal dengan galat "Cannot set properties of null" di Console.

Perbaikan: hapus seluruh baris `// --- INISIALISASI ---` beserta baris di bawahnya itu. Simpan, segarkan browser, buka Console (F12) — galat itu seharusnya hilang.

Temuan 2 — variabel CSS yang tidak pernah ada. Buka `css/style.css`, cari `var(--warna-header)`. Ada **dua** pemakaian (pada `.filter` dan `#form-status`), padahal `:root` hanya mendefinisikan `--warna-utama`, `--warna-kedua`, dan `--warna-aksen`. Variabel yang salah ketik membuat browser diam-diam mengabaikan aturan warna itu — tidak ada galat yang terlihat, hanya warna yang tidak sesuai rencana.

Perbaikan: ganti kedua `var(--warna-header)` menjadi `var(--warna-utama)` (variabel yang sama dipakai `.site-header`).

Temuan 3 — nama berkas yang menyesatkan. Periksa `img/produk.png` lewat **Properties/Info** sistem operasi Anda. Berkas ini sebenarnya berformat **AVIF**, hanya diberi akhiran `.png`. Browser modern umumnya masih menampilkannya, tapi ini kebiasaan penamaan yang keliru dan rapuh.

> ⚠ ***Bagian ini akan dijelaskan secara khusus oleh Instruktur.***

Perbaikan: jika tersedia, ekspor ulang gambar tersebut sebagai PNG sungguhan, lalu timpa `img/produk.png`. Jika belum sempat, catat ini di `README.md` sebagai utang teknis yang diketahui.

Materi 1: PHP vs JavaScript — Dua Tempat Kode Berjalan

Sepanjang Bagian A, seluruh kode JavaScript Anda berjalan di **browser pengguna**. Siapa pun bisa membuka DevTools dan membaca kode itu apa adanya — termasuk mengubah nilai langsung di Console. Untuk tampilan dan interaksi, ini wajar. Tapi untuk hal yang harus **dipercaya** (harga, stok, siapa yang sudah login), kode yang berjalan di browser tidak pernah cukup aman.

PHP berjalan di server — komputer yang menjalankan Laragon. Prosesnya:

```
Browser --- permintaan (request) ---> Server (Laragon: Apache + PHP)
                                          |
                                          | PHP dijalankan DI SINI
                                          | (berkas data dibaca/ditulis, hasil disiapkan)
                                          v
Browser <--- balasan (response, berupa HTML/JSON) ---
```

Peserta/pengguna **tidak pernah melihat kode PHP itu sendiri** — yang sampai ke browser hanyalah *hasil* dari kode itu. Inilah sebabnya logika yang harus dipercaya ditaruh di PHP, bukan JavaScript — poin ini akan sangat penting di Modul 5F saat merancang checkout.

Proses	**JavaScript (Bagian A)**	**PHP (Bagian B2)**
Berjalan di	Browser pengguna	Server (Laragon)
Terlihat oleh pengguna?	Ya, lewat DevTools	Tidak — hanya hasilnya
Butuh server untuk jalan?	Tidak (`file://` cukup)	**Ya, selalu**
Akhiran berkas	`.js`	`.php`
Bisa menulis balik ke berkas data?	**Tidak** (browser tidak bisa menyimpan ke berkas proyek)	**Ya** — inilah salah satu alasan utama butuh PHP

Materi 2: Sintaks Dasar PHP

Kode PHP ditulis di antara tag `<?php` dan `?>`, biasanya di dalam berkas `.php` yang juga boleh berisi HTML biasa.

```
<?php
echo "Halo, Kopi Nusantara!";
?>
```

Buat `latihan-php/halo.php` berisi kode di atas, lalu buka lewat **`http://localhost/.../latihan-php/halo.php`** (bukan dobel klik).

Variabel. Nama variabel **selalu diawali tanda dolar (`$`)**, dan PHP tidak membedakan `const`/`let`:

```
<?php
$namaProduk = "Kopi Gayo";
$harga = 80000;

echo "Produk: " . $namaProduk . ", harga: Rp " . $harga;
// atau dengan interpolasi string (hanya jalan di tanda kutip GANDA):
echo "Produk: $namaProduk, harga: Rp $harga";
```

JavaScript	PHP	Catatan
`const nama = "...";`	`$nama = "...";`	PHP tidak punya `const` untuk variabel biasa
`nama + " " + lain`	`$nama . " " . $lain`	PHP memakai **titik** (`.`) untuk menggabung teks
`` `Halo ${nama}` ``	`"Halo $nama"`	Interpolasi PHP hanya jalan di tanda kutip **ganda**
`typeof nilai`	`gettype($nilai)`	Memeriksa tipe data

Array asosiatif adalah padanan PHP untuk objek JavaScript:

```
<?php
$produk = [
    "slug"  => "toraja",
    "nama"  => "Kopi Toraja",
    "harga" => 85000,
];

echo $produk["nama"];     // Kopi Toraja
$produk["harga"] = 90000; // mengubah nilai
```

Array berindeks dan array asosiatif bisa digabung menjadi **array bersarang** — bentuk yang akan sangat sering dipakai di modul ini, karena **inilah bentuk alami sebuah berkas JSON** setelah dibaca PHP:

```
<?php
$daftarProduk = [
    ["slug" => "toraja", "nama" => "Kopi Toraja", "harga" => 85000],
    ["slug" => "gayo",   "nama" => "Kopi Gayo",   "harga" => 80000],
];

echo $daftarProduk[0]["nama"];   // Kopi Toraja
```

Materi 3: Percabangan, Perulangan, dan Fungsi

```php
<?php
$total = 150000;

if ($total >= 200000) {
    echo "Diskon 10%";
} elseif ($total >= 100000) {
    echo "Diskon 5%";
} else {
    echo "Tidak ada diskon";
}
```

Perulangan memakai `foreach` untuk menelusuri array — akan terus-menerus dipakai sepanjang modul ini untuk menelusuri data yang dibaca dari berkas JSON:

```php
<?php
$kategori = ["arabika", "robusta"];

foreach ($kategori as $k) {
    echo $k . "\n";
}

$produk = ["nama" => "Kopi Toraja", "harga" => 85000];
foreach ($produk as $kunci => $nilai) {
    echo "$kunci: $nilai\n";
}
```

Fungsi ditulis dengan kata kunci `function`, parameter boleh diberi tipe data:

```php
<?php
function hitungTotal(int $harga, int $jumlah): int {
    return $harga * $jumlah;
}

echo hitungTotal(85000, 2);   // 170000
```

Latihan 1

1. Buat `latihan-php/variabel.php`: deklarasikan tiga variabel (nama produk, harga, kategori), tampilkan dengan `echo` memakai interpolasi string.
2. Buat `latihan-php/array.php`: buat array asosiatif untuk satu produk Kopi Nusantara, tampilkan `nama` dan `harga`-nya.
3. Buat `latihan-php/perulangan.php`: buat array berisi 3 nama kategori, tampilkan masing-masing dengan `foreach`, diberi nomor urut.
4. Buat fungsi `formatRupiah(int $angka): string` yang mengembalikan teks berawalan `"Rp "` (petunjuk: `number_format($angka, 0, ",", ".")`).
5. *Tantangan:* buat array bersarang berisi 3 produk (tiap produk: `slug`, `nama`, `harga`), lalu tampilkan nama dan harga ketiganya memakai `foreach`.

Rangkuman KB 4F.1

- PHP berjalan di server, hasilnya (bukan kodenya) yang sampai ke browser — dan hanya PHP (bukan JavaScript murni) yang bisa menulis balik ke berkas data proyek.
- Variabel PHP diawali `$`, penggabungan teks memakai `.`, interpolasi hanya di tanda kutip ganda.
- Array asosiatif dan array bersarang adalah bentuk data yang akan terus dipakai sepanjang Bagian B2, karena mencerminkan bentuk JSON itu sendiri.

Cek Kemampuan KB 4F.1

- ☐ Ketiga temuan di proyek Kopi Nusantara sudah diperbaiki dan diuji ulang.
- ☐ Saya dapat menjelaskan mengapa menulis balik ke berkas data hanya bisa dilakukan PHP, bukan JavaScript murni.
- ☐ Saya dapat menulis variabel, array asosiatif, `foreach`, dan fungsi PHP tanpa melihat contoh.

Kesalahan umum: lupa tanda `$` di depan nama variabel; memakai `+` alih-alih `.` untuk menggabung teks; menulis interpolasi di tanda kutip tunggal; lupa menjalankan lewat `http://localhost/...`.